

Performance Report

C++ Group Chat System with Audio/Video, Caching, and OS Scheduling

CS375 Operating Systems Final Project

Prepared for: Luvell Glanton | Date: May 6, 2026

1. Executive Summary

This report evaluates the performance of the GroupChat C++ client-server program. The program supports TCP text chat, multiple chat groups, group switching, recent-message history, a fixed-size cache, a binary frame protocol, logging, and optional audio/video transfer. The server uses a main accept loop, one reader thread per connected client, and a 4-worker thread pool to process queued chat tasks.

The included bot test passed all feature checks. In the timing benchmark, the server was tested locally with 5 clients and 100 messages. Round Robin/FIFO mode reached 13.7 messages per second with 48.36 ms average sender-observed latency. SJF mode reached 14.7 messages per second with 45.50 ms average latency.

2. Program Overview

Component	Performance Role
TCP server socket	Accepts client connections and relays framed messages over localhost or network sockets.
Client reader threads	Each connected client has a server-side read loop that receives frames and submits text work.
Thread pool	Uses 4 worker threads and a condition variable so workers sleep when no chat tasks are queued.
Scheduling mode	RR uses FIFO order; SJF uses message length as the task cost and handles shorter text messages first.
GroupManager mutex	Protects shared group membership, client IDs, cache access, and log appends.
Message cache	Stores the latest 20 messages per group using circular-buffer style eviction.
Binary frame protocol	Sends 1 byte frame type, 4 byte payload length, and payload data for text and media chunks.
Logging	Appends formatted messages to logs/chat_log.txt with timestamp, group, sender, and message text.

3. Test Environment and Method

Build and test environment: Linux container using CMake, GNU C++17, POSIX sockets, std::thread, std::mutex, and std::condition_variable.

Build command used for verification:

```
cmake -S . -B build_perf && cmake --build build_perf -j2
```

Functional test command:

```
./build_perf/groupchat_server 5555 rr      # Terminal 1
./build_perf/bot_test                      # Terminal 2
```

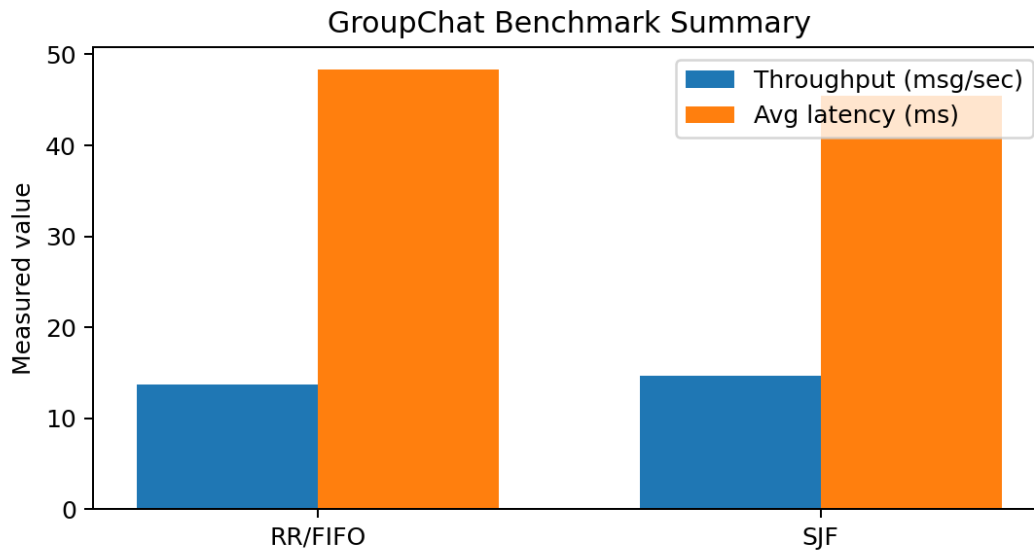
Benchmark method: a Python socket test connected 5 clients to the server, joined them to one group, sent 100 alternating short and longer text messages, and measured time from send to the sender receiving its own broadcast back from the server. This is application-level localhost latency, not real internet latency.

4. Benchmark Results

Mode	Throughput	Average Latency	Median Latency	P95 Latency	Max Latency
RR/FIFO	13.7 msg/sec	48.36 ms	43.743 ms	99.305 ms	198.275 ms
SJF	14.7 msg/sec	45.50 ms	43.703 ms	97.876 ms	196.246 ms

Shared benchmark setup: 5 clients, 100 messages, localhost TCP, mixed short/long text payloads.

Result interpretation: SJF was slightly faster in this test because short messages could move ahead of longer messages in the priority queue. The difference was small because most cost came from socket I/O, mutex-protected group operations, message formatting, and file logging rather than heavy CPU work.



5. Required Performance Metrics

Metric	Observed Result	Analysis
Message rate	RR: 13.7 msg/sec; SJF: 14.7 msg/sec	SJF improved throughput by about 7.3% in the local test. Either mode is usable for normal chat messages.
Latency	RR avg: 48.36 ms; SJF avg: 45.50 ms	Both modes stayed under 50 ms average local latency. The p95 values near 98-99 ms show occasional queueing or socket delay.
Cache hit/miss	Cache behavior exists, but hit/miss counters are not instrumented.	A hit can be defined as a joining user receiving stored group history. A miss is when a group has no history. Add counters for exact reporting.
Thread usage	About 10 application threads in the 5-client benchmark: 1 main/accept thread, 5 client reader threads, and 4 worker threads.	The fixed worker pool scales better than creating a worker for every message, but reader threads still grow linearly with client count.
Paging faults	No application-level page-fault counter or simulated page-fault module is included.	The OS handled real paging automatically. Add getrusage() counters or a simulated fixed memory pool with LRU page eviction.

6. Bottlenecks and Performance Risks

- Single GroupManager mutex: group membership, message cache updates, and log writes share one lock. This is safe, but it can become a bottleneck when many clients send at the same time.
- File logging is synchronous: every broadcast appends to logs/chat_log.txt. Disk I/O can slow down message processing under heavier loads.
- One server reader thread per client: this is fine for a class demo, but a select/poll/epoll design would scale better for very high client counts.
- Media transfer uses 4096-byte chunks. Large videos create many frames and many send calls.

- The current video send function sends VIDEO_BEGIN and VIDEO_CHUNK frames but does not send VIDEO_END. The relay logic supports VIDEO_END, but the /video command path should also send it so receivers close the file and update /play correctly.

7. Operating System Concepts Shown by the Program

OS Concept	How the Program Demonstrates It
Networking and I/O	TCP sockets use send and recv through a custom binary frame protocol.
Concurrency	Multiple clients connect at once while reader threads and worker threads run concurrently.
Scheduling	The thread pool chooses work using FIFO/RR behavior or SJF based on message length.
Synchronization	std::mutex protects shared group state, and std::condition_variable prevents busy-waiting.
Caching	Each group keeps a fixed-size recent message history and evicts the oldest entry after 20 messages.
File systems	The server appends persistent chat history to logs/chat_log.txt.
Memory management	Payloads use std::vector/std::string and MAX_FRAME_SIZE limits oversized frames.

8. Recommendations for Improvement

1. Add counters for total messages, bytes, cache history hits, cache misses, and evictions. Print them with a /stats command or when the server exits.
2. Add getrusage() or /proc/self/stat instrumentation for minor faults, major faults, CPU time, and memory usage.
3. Move logging into a separate logger thread and queue so broadcast latency is not blocked by disk writes.
4. Use a per-group mutex instead of one global GroupManager mutex so unrelated groups can broadcast in parallel.
5. Fix send_video_file() by sending FRAME_VIDEO_END after all chunks are sent.
6. Add a repeatable benchmark target, such as ./build_perf/perf_test, for demo-ready results.

9. Reproducibility Checklist

- Build the project with CMake.
- Run bot_test and capture the PASS all bot feature checks output.
- Run the benchmark in RR and SJF mode and record throughput/latency numbers.
- Show logs/chat_log.txt after sending messages.
- Show a client joining a group and receiving recent history.
- Show an audio/video transfer test, then use /play or show the downloaded file.

11. Conclusion

The GroupChat program performs well for a small operating-systems final project demo. It passes the included functional test and shows the main OS ideas: sockets, threads, scheduling, synchronization, caching, and file I/O. The measured local benchmark showed usable latency in both scheduling modes, with SJF slightly ahead of RR/FIFO for the tested mixed message sizes. The main performance improvements would be adding formal counters, reducing lock contention, buffering logging, and completing the video end-frame path.

Source Basis

This report was written from the uploaded GroupChat.zip source code, the uploaded CS375 final project description, the project README, the included bot_test.cpp output, and a small local benchmark run against the compiled server.